SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Constituent of Symbiosis International (Deemed University), Pune

Name of Student: Postiwala Vatsal Dipakbhai	PRN: 26070122509
Subject: Web Application and Development	Sem: III AY: 2026-27
EXPERIMENT No: 8	Assignment No: 8
TITLE: Develop a Node.js application that automatically sends an email to a specified recipient using the Nodemailer module and Gmail SMTP service.	

Create a Node.js application that automates sending emails

Course Outcome	CO3	Teaching Hours	2

Objective

To use the Nodemailer npm package to send automated emails (with attachments) from a Node.js application via an SMTP provider such as Gmail.

Theory

Nodemailer is the most widely used Node.js module for sending emails, supporting SMTP transport, HTML content, attachments and templating. Sending via a real provider (e.g. Gmail SMTP) requires authentication; since Google blocks plain-password SMTP logins for security, an App Password (a 16-character generated password tied to your Google account, requiring 2-Step Verification to be enabled) must be used instead of your real account password. Never hardcode credentials in source code — use environment variables (a .env file with the dotenv package) so secrets are not committed to version control.

Software / Hardware Requirements

Node.js, npm packages: nodemailer, dotenv, a Gmail account with 2-Step Verification and an App Password generated.

Step-by-Step Procedure

1. Create project folder lab9_email, run npm init -y, then install dependencies.

```
npm install nodemailer dotenv
```

2. On your Google account, enable 2-Step Verification (myaccount.google.com/security), then generate an App Password (Security > App passwords > select 'Mail' > Generate) and copy the 16-character password.

3. Create a .env file in the project root to store credentials securely (never commit this file to Git

— add it to .gitignore).

```
EMAIL_USER=youraddress@gmail.com
EMAIL_PASS=your16charapppassword
```

4. Create sendmail.js. Load environment variables with dotenv and configure a Nodemailer transporter using Gmail's SMTP service.

```
// sendmail.js
require('dotenv').config();
const nodemailer = require('nodemailer');

const transporter = nodemailer.createTransport({
  service: 'gmail',
  auth: {
    user: process.env.EMAIL_USER,
    pass: process.env.EMAIL_PASS
  }
});
```

5. Define a mail options object with recipient, subject, plain-text and HTML body, then call transporter.sendMail().

```
const mailOptions = {
  from: process.env.EMAIL_USER,
  to: 'recipient@example.com',
  subject: 'Automated Email from Node.js Lab',
  text: 'This is a plain-text fallback message.',
  html: '<h2>Hello!</h2><p>This email was sent automatically using
<b>Nodemailer</b> from a Node.js application.</p>'
};

transporter.sendMail(mailOptions, (err, info) => {
  if (err) {
    console.error('Error sending email:', err);
  } else {
    console.log('Email sent successfully:', info.response);
  }
});
```

6. Run node sendmail.js and check the console for the success message, then check the recipient inbox (and spam folder) to confirm delivery.

7. Add an attachment to the email by including an attachments array in mailOptions, pointing to a local file.

```
mailOptions.attachments = [
  { filename: 'report.pdf', path: __dirname + '/report.pdf' }
];
```

8. Re-run the script and confirm the recipient receives the email with the attached file.

9. Wrap the send logic into a simple HTTP server with a form, so an email can be triggered by submitting a webpage rather than by running the script directly (combining Experiment 7's http module with Nodemailer).

```
const http = require('http');
const server = http.createServer((req, res) => {
  if (req.url === '/' ) {
    res.setHeader('Content-Type', 'text/html');
    res.end('<form method="POST" action="/send"><input name="to"
placeholder="recipient email"><button>Send Test Mail</button></form>');
  } else if (req.url === '/send' && req.method === 'POST') {
    let body = '';
    req.on('data', chunk => body += chunk);
    req.on('end', () => {
      const to = decodeURIComponent(body.split('=')[1]);
      transporter.sendMail({ ...mailOptions, to }, (err) => {
        res.end(err ? 'Failed to send' : 'Email sent to ' + to);
      });
    });
  }
});
server.listen(3003, () => console.log('Mail server on 3003'));
```

Output :

```
PS D:\SIT\WAD\javascript> cd lab8_email
PS D:\SIT\WAD\javascript\lab8_email> npm init -y
Wrote to D:\SIT\WAD\javascript\lab8_email\package.json:

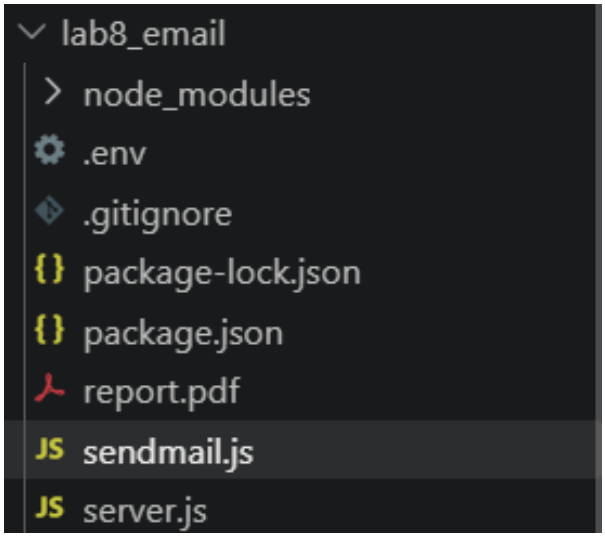
{
  "name": "lab8_email",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}

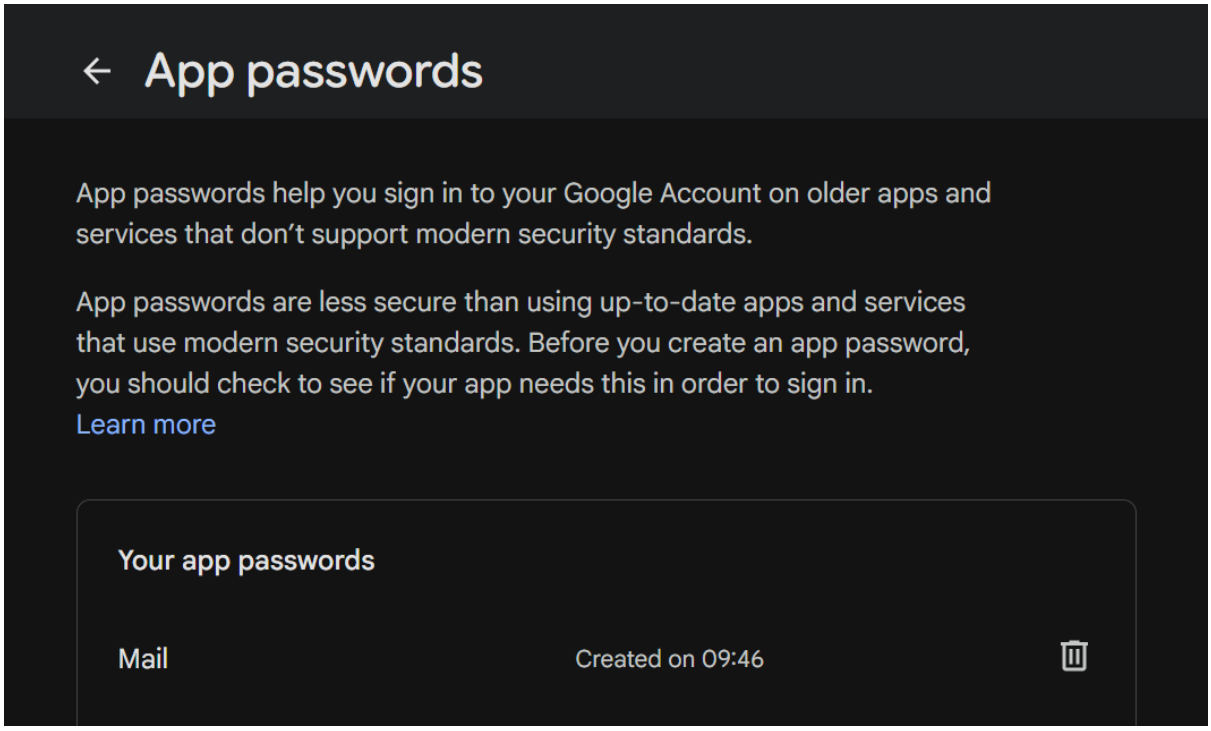

PS D:\SIT\WAD\javascript\lab8_email> npm install nodemailer dotenv

added 2 packages, and audited 3 packages in 904ms

1 package is looking for funding
  run `npm fund` for details

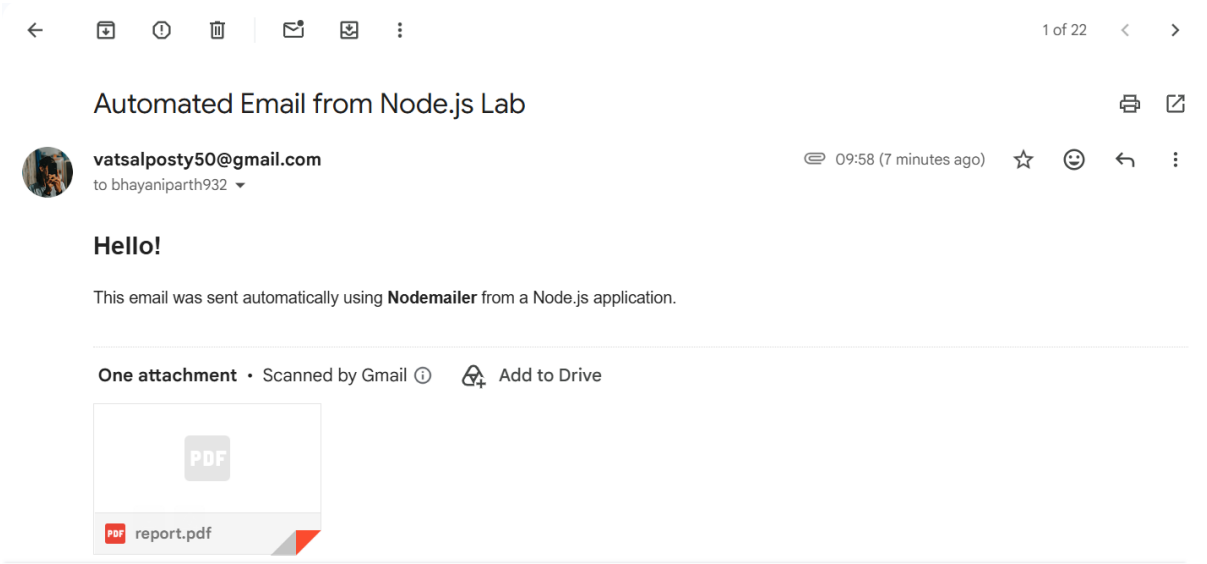
found 0 vulnerabilities
```



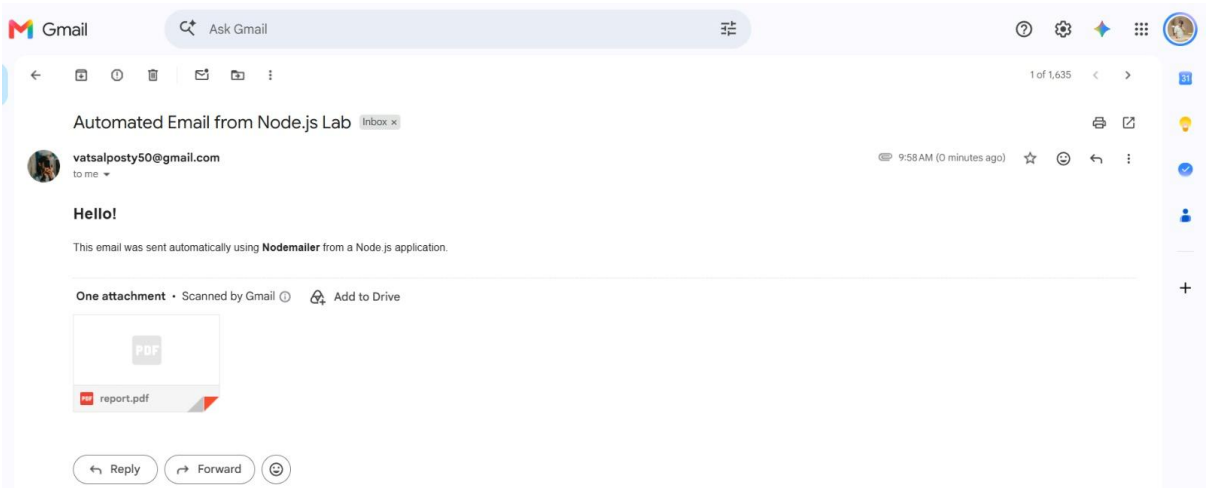



```
PS D:\SIT\WAD\javascript\lab8_email> node sendmail.js
◇ injected env (3) from .env
Email sent successfully: 250 2.0.0 OK  1790051312 5a478bee46e88-33e626de382sm1311894eec.0 - gsmtp
PS D:\SIT\WAD\javascript\lab8_email> x
```

Sender side :



Receiver Side :



Conclusion:

In this experiment, a Node.js application was successfully developed to automate email dispatch using the Nodemailer package and Gmail SMTP service. Security best practices were implemented by isolating credentials in environment variables with dotenv and authenticating via Google App Passwords. The implementation successfully demonstrated sending rich HTML emails along with PDF file attachments using both direct terminal execution and a web-based HTTP server interface.